Kruskals Algorithm

AlgoExpert · Hard · Famous Algorithms

The problem

- Undirected weighted graph as adjacency list ([dest, weight]); return its minimum spanning tree (forest) in the same format.
- Sort edges + union-find.

Intuition

- Add edges from lightest to heaviest, skipping any that would form a cycle, detected with union-find.

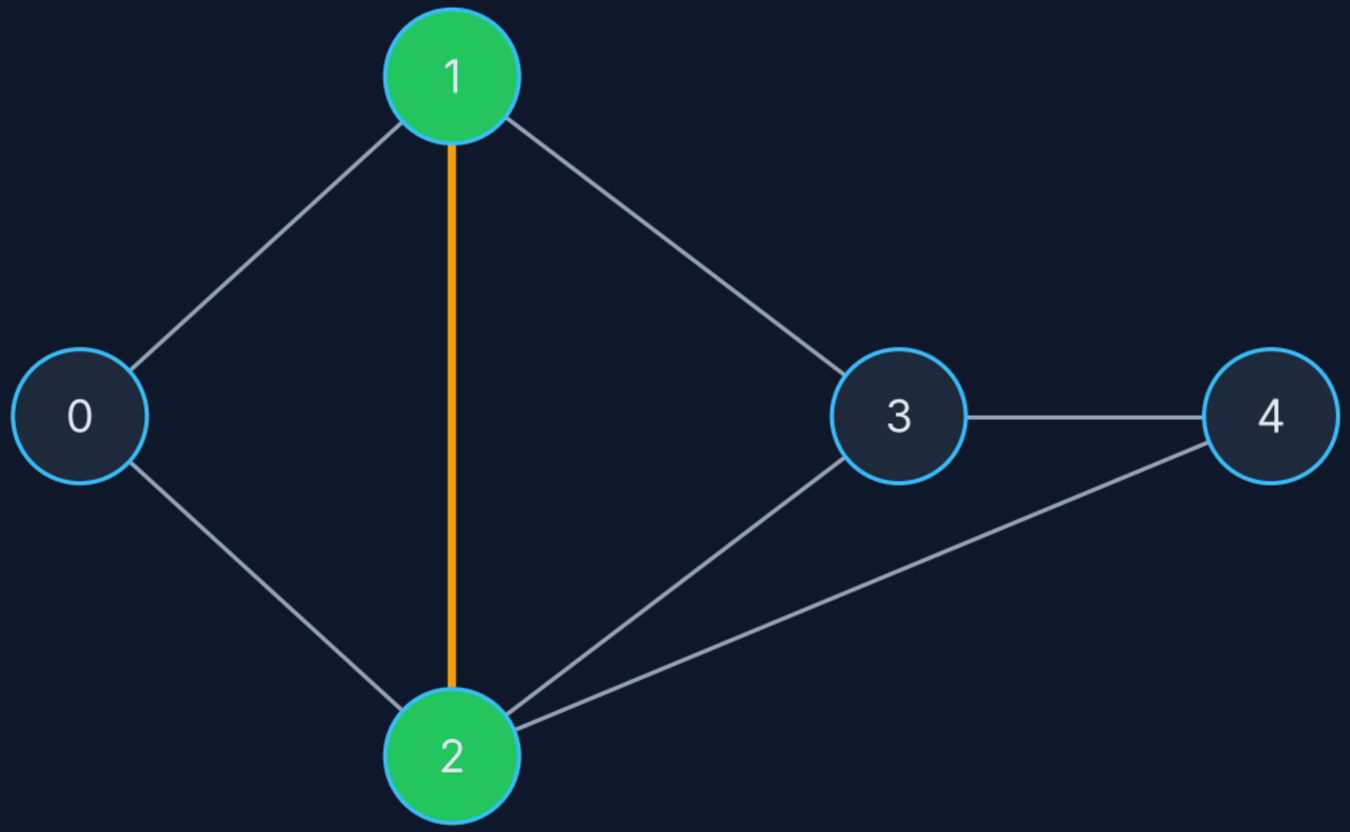
Brute force vs optimal

- Try all spanning trees → exponential
- Kruskal: sort edges + union-find → O(E log E)
- Prim: grow from a vertex with a heap

Approach

- 1. Collect and sort edges by weight
- 2. Union-find over vertices
- 3. Add edges joining different sets
- 4. Return as adjacency list

Worked example



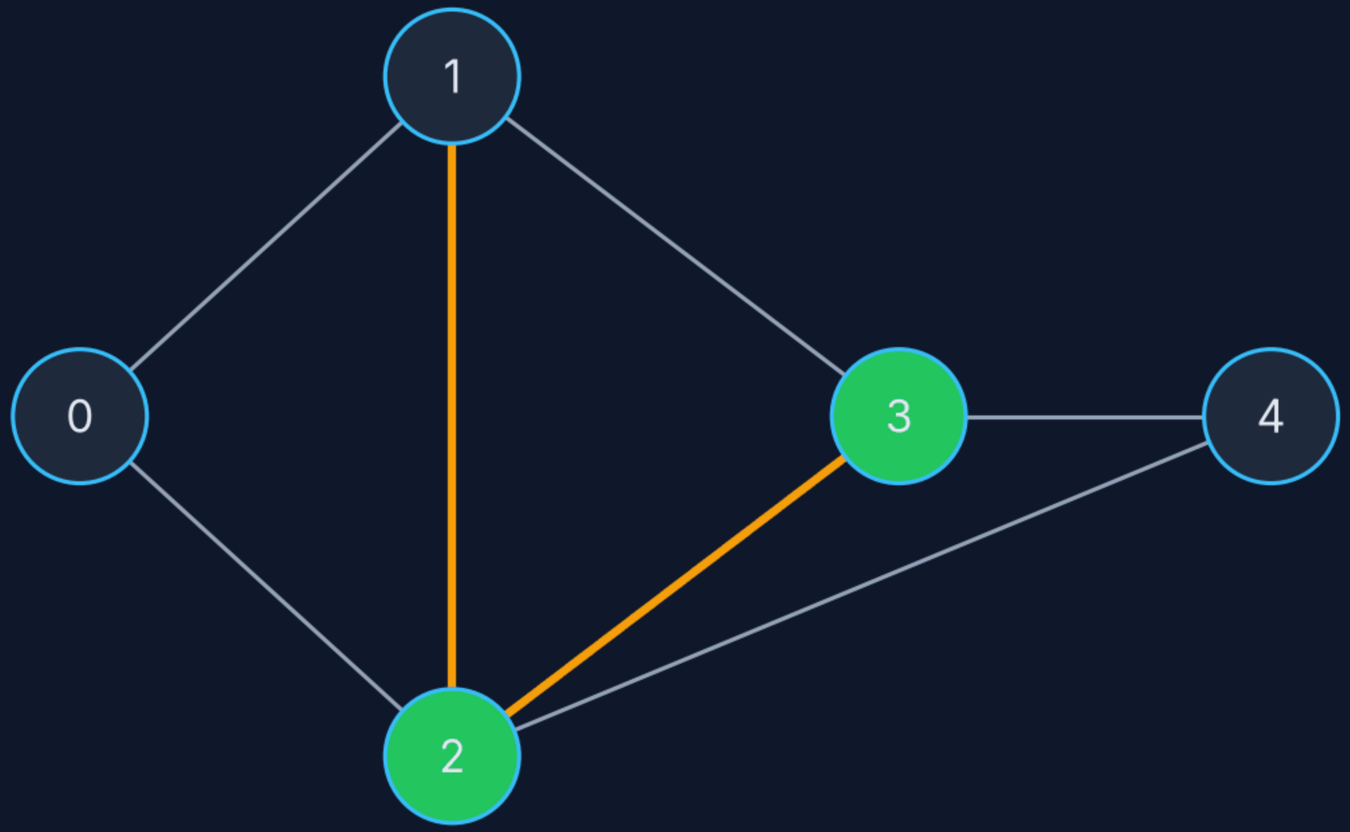

```
edge 1–2 (w 1)

add ✓

MST weight 1
```

Edge 1–2 (1): add

Worked example



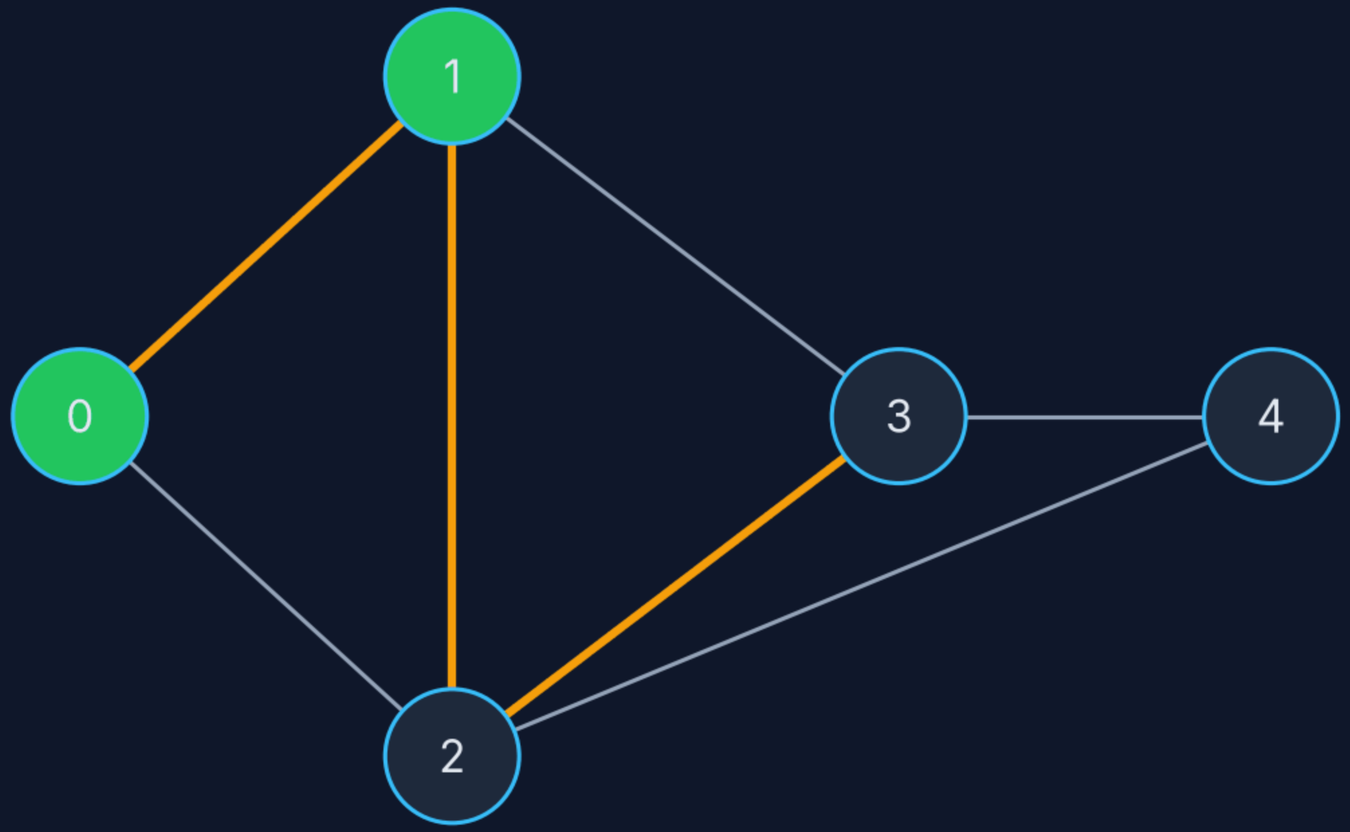
edge 2–3 (w 2)

add ✓

MST weight 3

Edge 2–3 (2): add

Worked example



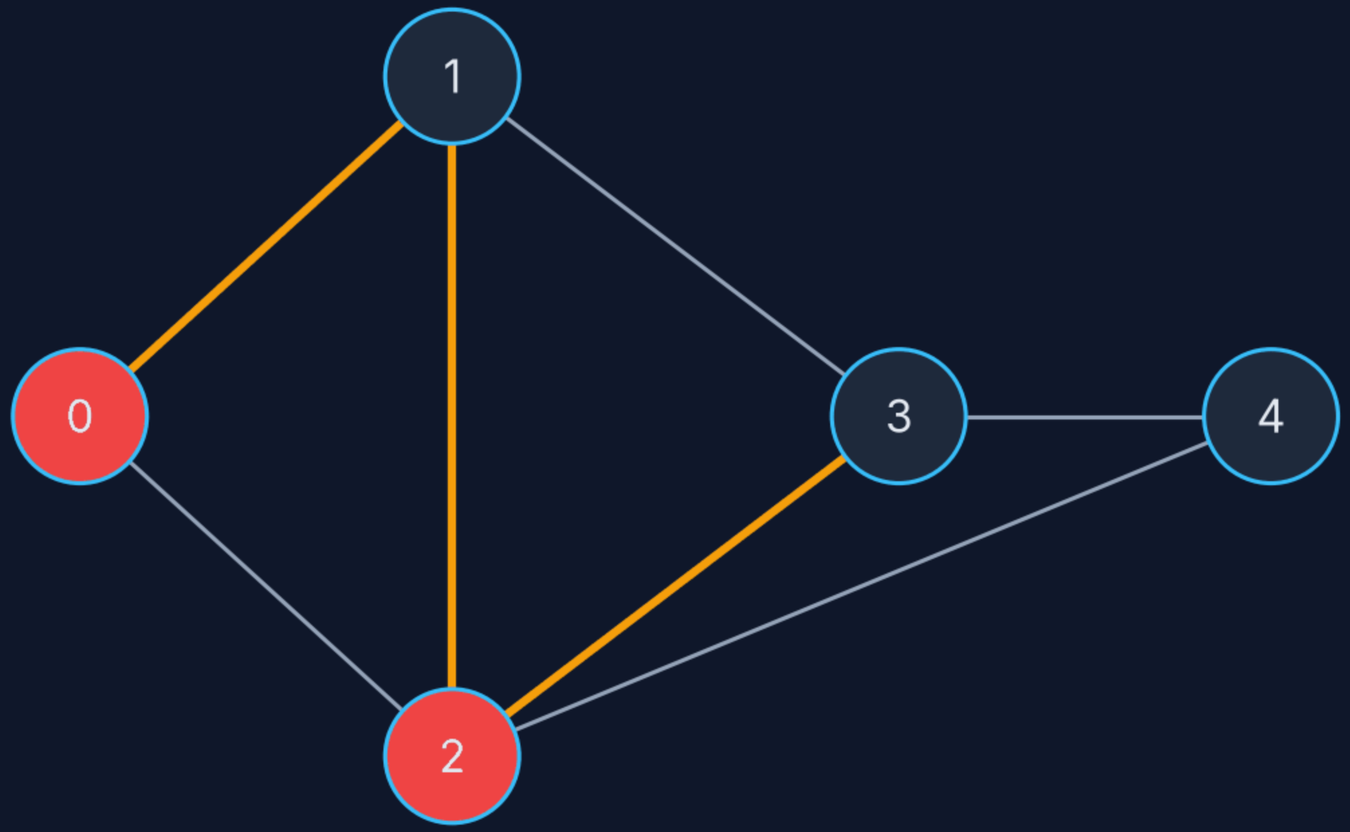

```
edge 0–1 (w 3)

add ✓

MST weight 6
```

Edge 0–1 (3): add

Worked example



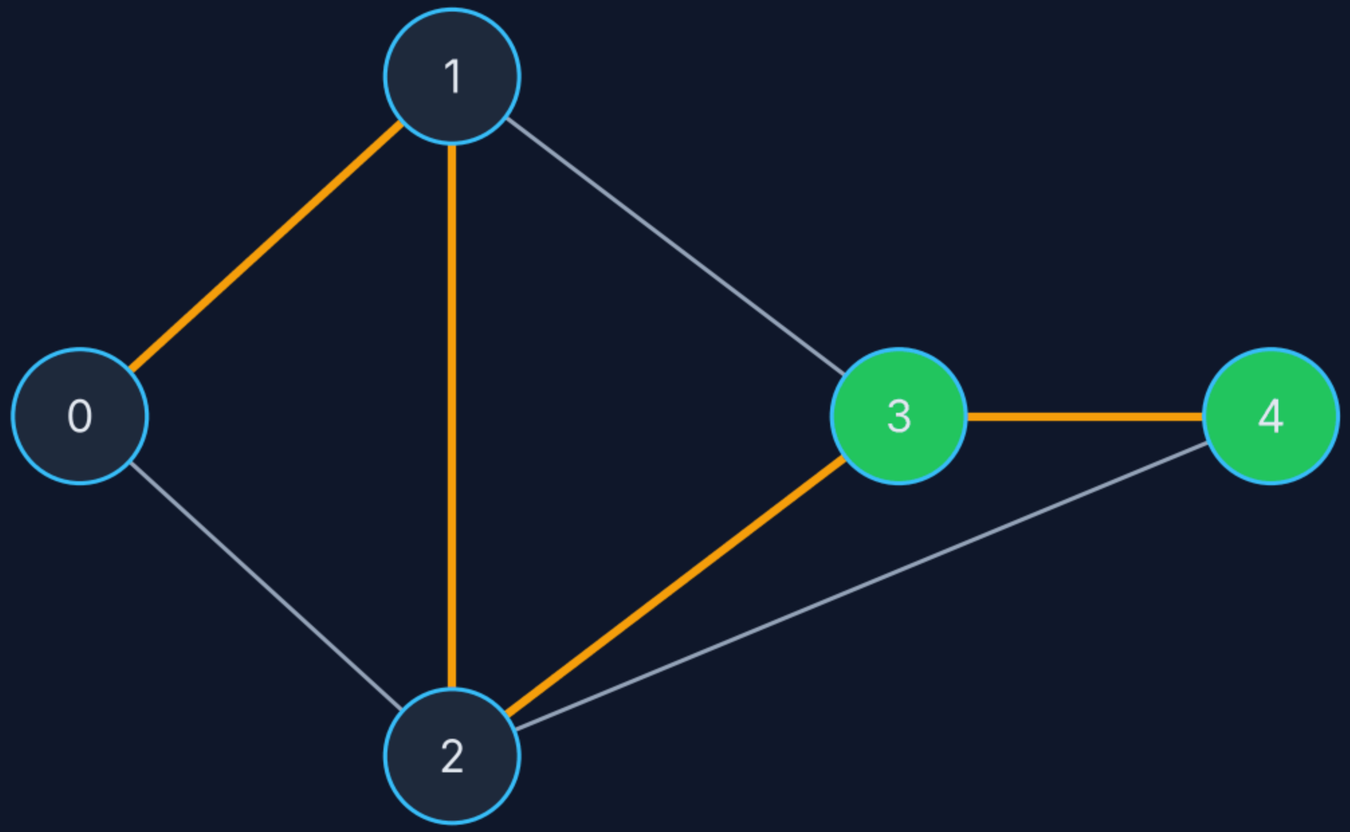
edge 0–2 (w 5)

would form a cycle ✗

MST weight 6

Edge 0–2 (5): skip (cycle)

Worked example



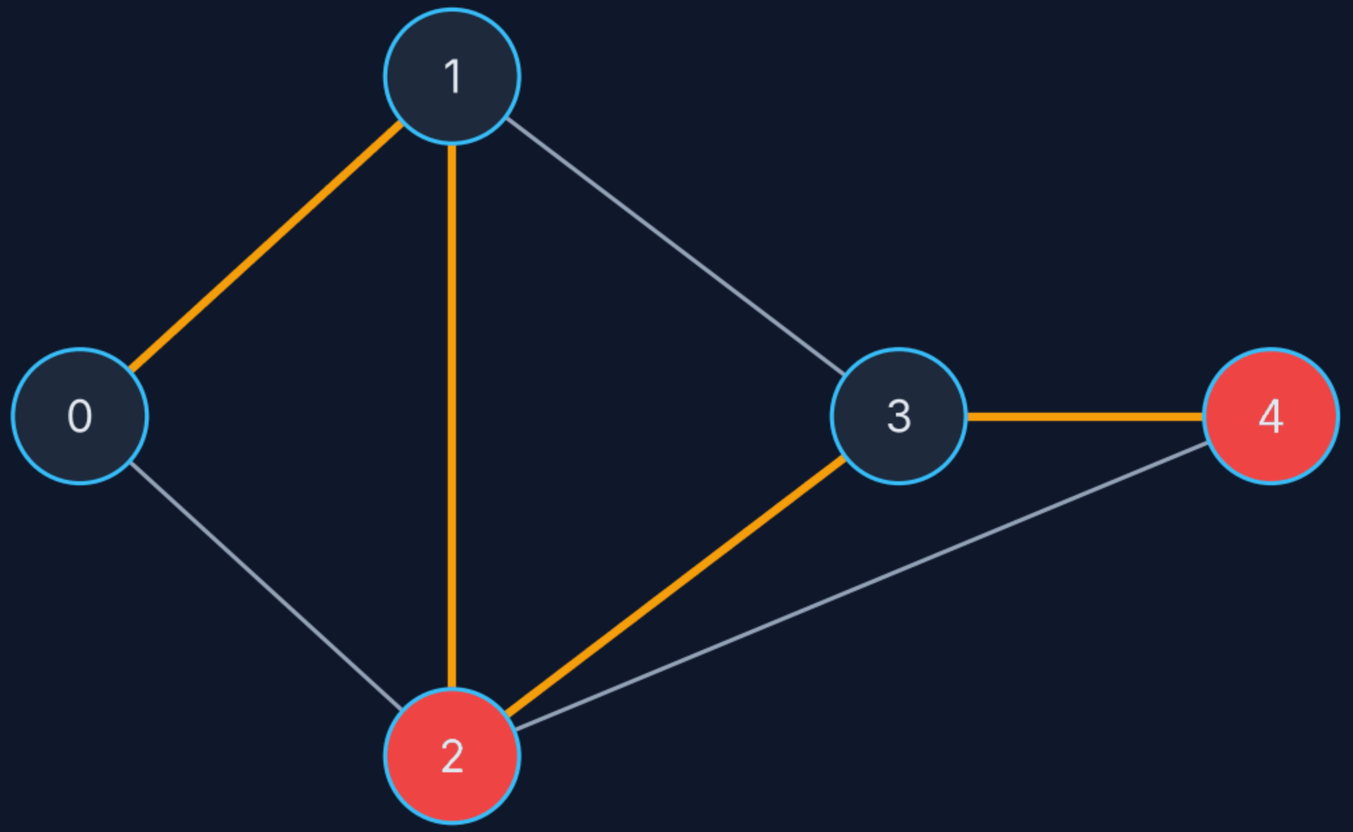

```
edge 3–4 (w 6)

add ✓

MST weight 12
```

Edge 3–4 (6): add

Worked example



edge 2–4 (w 7)

would form a cycle ✗

MST weight 12

Edge 2–4 (7): skip (cycle)

Java solution (1/2)

```java
public class KruskalsAlgorithm {
    public static int[][][] kruskalsAlgorithm(int[][][] edges) {
        int n = edges.length;
        List<int[]> all = new ArrayList<>();
        for (int u = 0; u < n; u++) for (int[] e : edges[u]) if (u < e[0]) all.add(new int[]{u, e[0], e[1]});
        all.sort(Comparator.comparingInt(e -> e[2]));
        int[] parent = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
        List<List<int[]>> mst = new ArrayList<>();
        for (int i = 0; i < n; i++) mst.add(new ArrayList<>());
        for (int[] e : all) {
            int a = find(parent, e[0]), b = find(parent, e[1]);
            if (a == b) continue;
            parent[a] = b;
            mst.get(e[0]).add(new int[]{e[1], e[2]});
            mst.get(e[1]).add(new int[]{e[0], e[2]});
        }
        int[][][] out = new int[n][][];
        for (int i = 0; i < n; i++) out[i] = mst.get(i).toArray(new int[0][]);
        return out;
    }
```

Java solution (2/2)

```
    private static int find(int[] p, int x) { while (p[x] != x) { p[x] = p[p[x]]; x = p[x]; } return x; }
}
```

Complexity

- Time: O(e log e)
- Space: O(v + e)

Edge cases & pitfalls

- Undirected edges appear twice — dedupe
- Disconnected graph → spanning forest
- Use path compression

Interview tips & follow-ups

- Cut property explains correctness
- Kruskal vs Prim trade-offs

Recap

- Add edges from lightest to heaviest, skipping any that would form a cycle, detected with union-find.
- Complexity: O(e log e)